Proxmox VE Security

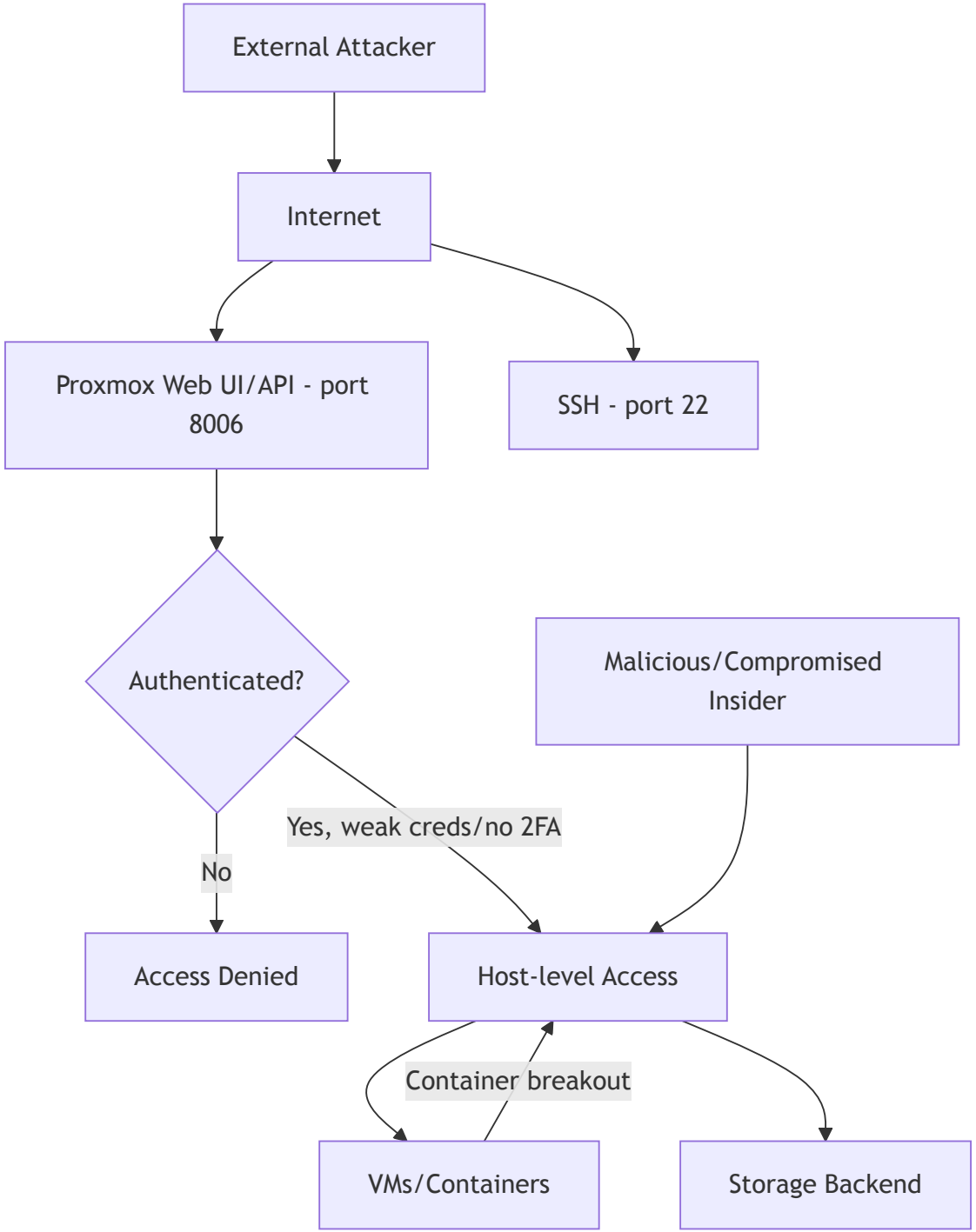
Day 1: Hardening & Access Control

Day 1 — Agenda

1. Course Introduction & Threat Model
2. Proxmox Security Architecture Overview
3. Host OS Hardening (Debian Baseline)
4. Authentication — PAM, Realms & Two-Factor Authentication
5. Role-Based Access Control (RBAC) Deep Dive
6. API Tokens & Least-Privilege Automation
7. Securing the Web UI & API Endpoints
8. TLS Certificates & Certificate Management
9. Auditing User Activity & Permissions
10. Lab: Hardening a Proxmox Node

1. Course Introduction & Threat Model

This course focuses on **securing** a Proxmox VE deployment: hardening, access control, encryption, logging, and compliance. It assumes familiarity with basic Proxmox VE administration.

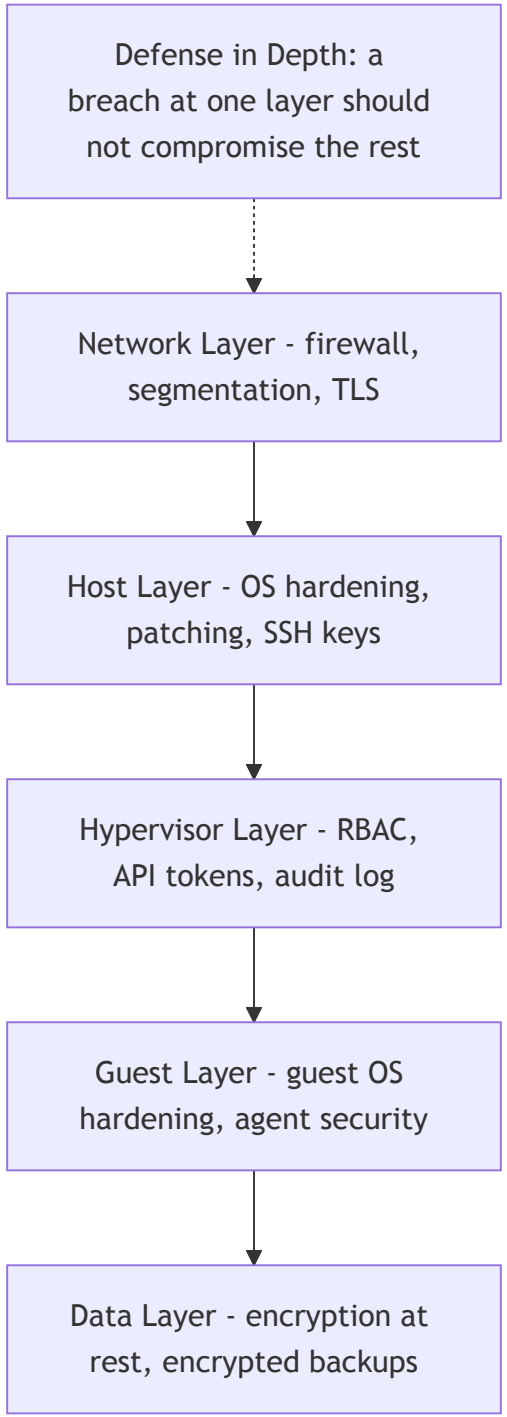

External Attacker
Internet
Proxmox Web UI/API - port 8006
SSH - port 22
Authenticated?
Malicious/Compromised Insider
Yes, weak creds/no 2FA
No
Access Denied
Host-level Access
Container breakout
VMs/Containers
Storage Backend

Key threat actors to design against:

- **External attackers** targeting the web UI/API or SSH
- **Malicious or compromised insiders** with legitimate credentials
- **Container/VM breakout** attempts from a compromised guest workload

2. Proxmox Security Architecture Overview

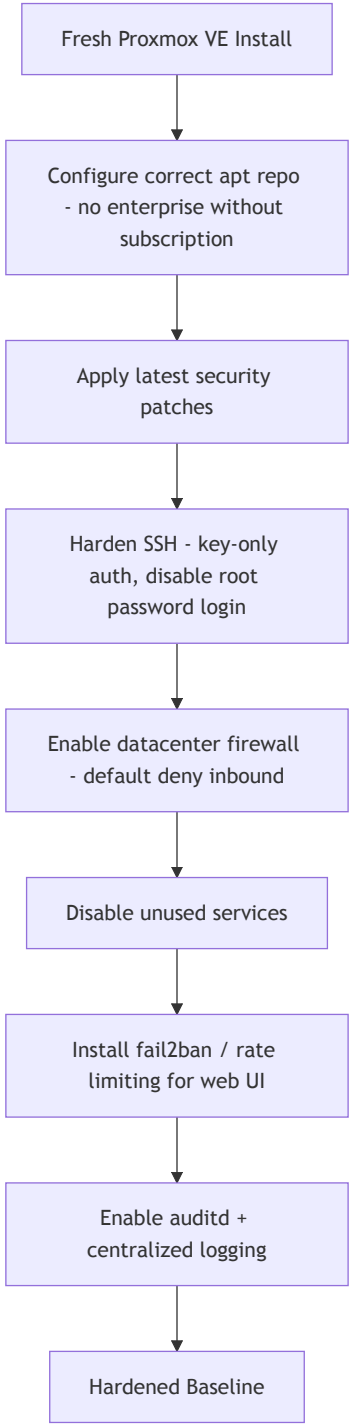
Security is not one control — it's layered so that a failure at one level doesn't compromise everything above or below it.


Defense in Depth: a breach at one layer should not compromise the rest
Network Layer - firewall, segmentation, TLS
Host Layer - OS hardening, patching, SSH keys
Hypervisor Layer - RBAC, API tokens, audit log
Guest Layer - guest OS hardening, agent security
Data Layer - encryption at rest, encrypted backups

This course works through each layer in order: host hardening and identity today, then network/storage/logging on Day 2 — reflecting how a real security architecture review is structured.

3. Host OS Hardening (Debian Baseline)

Proxmox VE runs on Debian — general Linux hardening practices apply, plus a few Proxmox-specific considerations.


Fresh Proxmox VE Install
Configure correct apt repo - no enterprise without subscription
Apply latest security patches
Harden SSH - key-only auth, disable root password login
Enable datacenter firewall - default deny inbound
Disable unused services
Install fail2ban / rate limiting for web UI
Enable auditd + centralized logging
Hardened Baseline

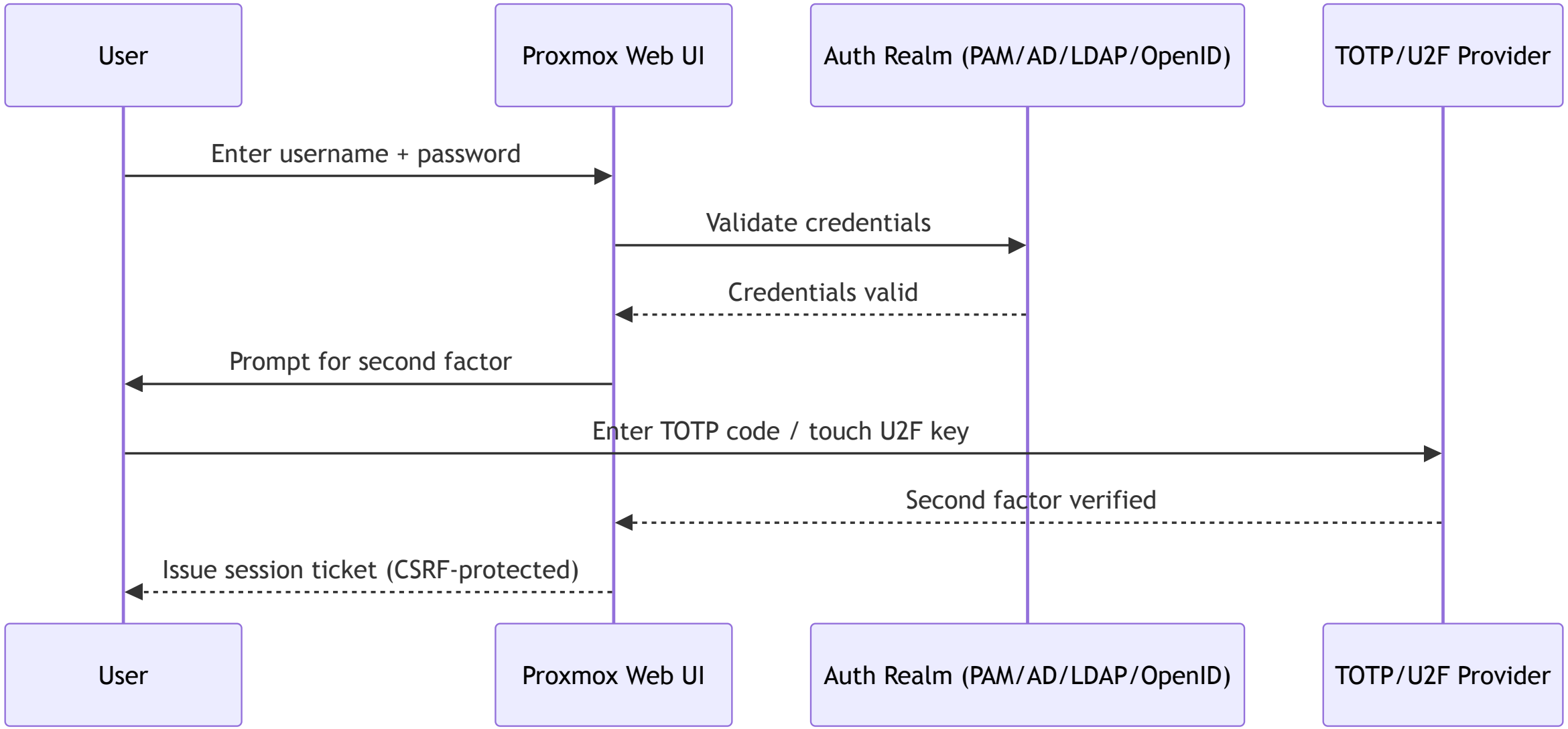
```
# SSH: key-only auth, no root password login
sed -i 's/^#PermitRootLogin.*/PermitRootLogin prohibit-password/' /etc/ssh/sshd_config
sed -i 's/^#PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
systemctl restart sshd

# Keep only the repos you actually need
rm -f /etc/apt/sources.list.d/pve-enterprise.list  # if unlicensed
apt update && apt full-upgrade -y
```

Avoid installing unrelated packages/services on hypervisor hosts — every extra service is additional attack surface on a system that ultimately controls all your VMs.

4. Authentication — PAM, Realms & Two-Factor Authentication

Proxmox VE supports multiple authentication **realms** (PAM, its own internal `pve` realm, LDAP, Active Directory, OpenID Connect) and layers **TOTP/U2F/WebAuthn** two-factor authentication on top of any of them.


User
Proxmox Web UI
Auth Realm (PAM/AD/LDAP/OpenID)
TOTP/U2F Provider
Enter username + password
Validate credentials
Credentials valid
Prompt for second factor
Enter TOTP code / touch U2F key
Second factor verified
Issue session ticket (CSRF-protected)
User
Proxmox Web UI
Auth Realm (PAM/AD/LDAP/OpenID)
TOTP/U2F Provider

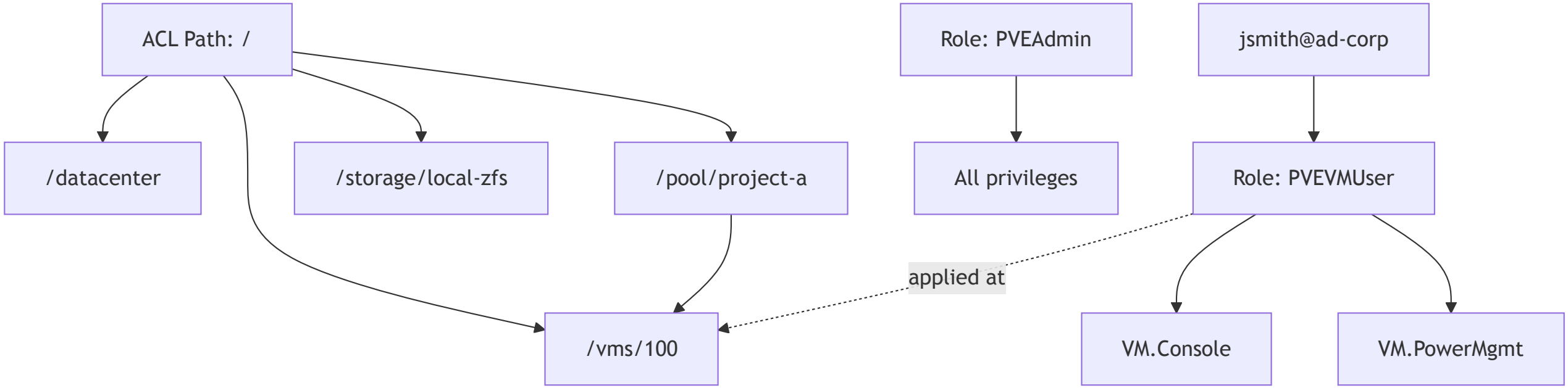
```
pveum realm add ad-corp --type ad --server ad.example.com --domain example.com

# Enforce TOTP for a realm
pveum realm modify ad-corp --tfa type=oath,digits=6,step=30
```

Never leave `root@pam` as the only administrative account with no 2FA — it's the single highest-value credential in the entire deployment.

5. Role-Based Access Control (RBAC) Deep Dive

Permissions combine three things: a **Role** (a bundle of privileges), an **ACL Path** (where in the resource tree it applies), and whether it **propagates** to child objects.

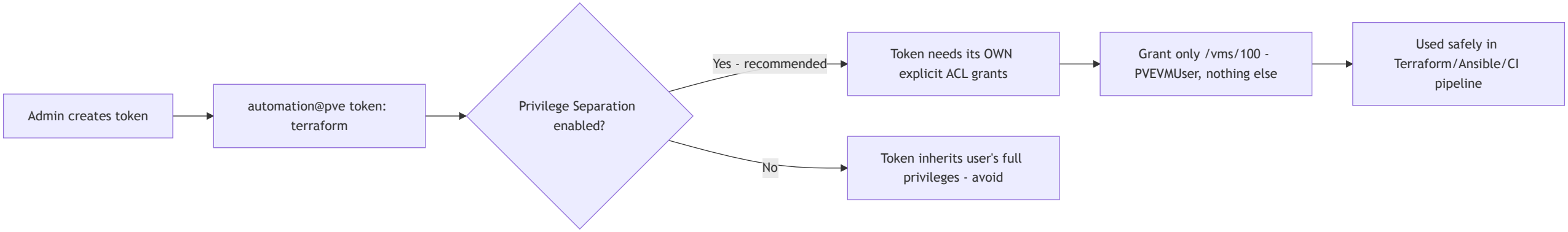

ACL Path: /
/datacenter
/storage/local-zfs
/pool/project-a
/vms/100
Role: PVEAdmin
All privileges
jsmith@ad-corp
Role: PVEVMUser
applied at
VM.Console
VM.PowerMgmt

```
# Custom role with only the privileges an operator actually needs
pveum role add VM-Operator -privs "VM.Console,VM.PowerMgmt,VM.Monitor"
pveum acl modify /vms/100 --users jsmith@ad-corp --roles VM-Operator
```

Avoid granting `Administrator` broadly. Build narrow custom roles per job function — this is the single highest-leverage RBAC practice for limiting blast radius.

6. API Tokens & Least-Privilege Automation

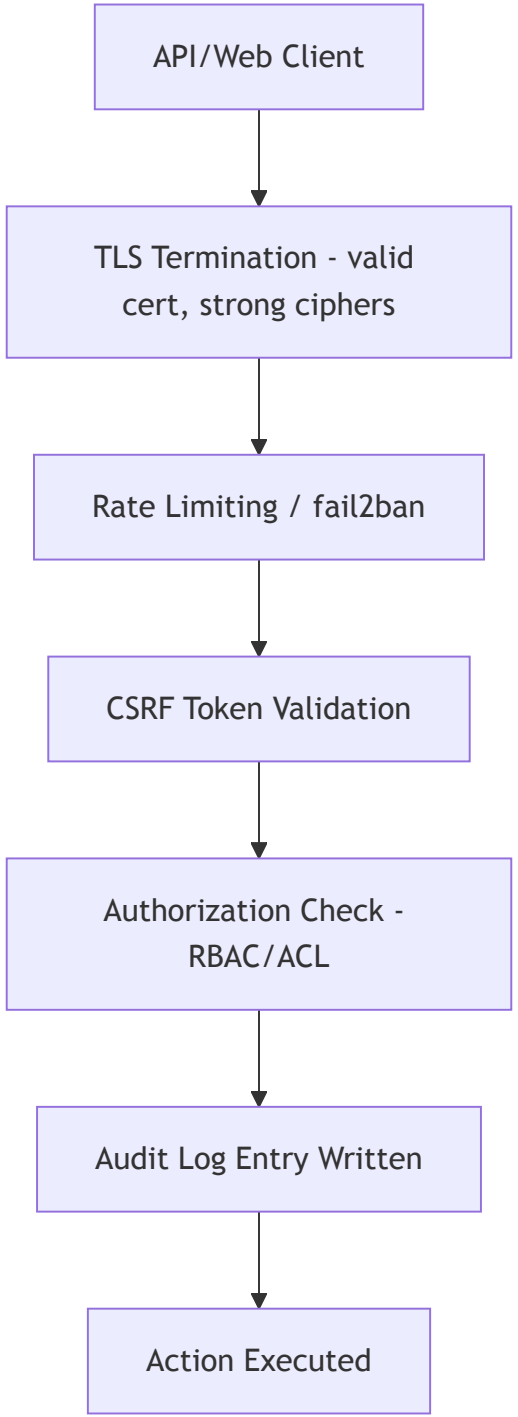
API tokens let automation (Terraform, Ansible, CI/CD) authenticate without embedding a user's real password anywhere.


Admin creates token
automation@pve token: terraform
Privilege Separation enabled?
Yes - recommended
No
Token needs its OWN explicit ACL grants
Token inherits user's full privileges - avoid
Grant only /vms/100 - PVEVMUser, nothing else
Used safely in Terraform/Ansible/CI pipeline

```
# Create a token WITH privilege separation (recommended)
pveum user token add automation@pve terraform --privsep 1

# Grant the token its OWN scoped ACL - it does NOT inherit the user's roles
pveum acl modify /vms/100 --tokens 'automation@pve!terraform' --roles VM-Operator
```

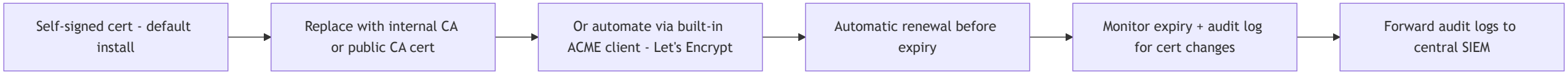
Store tokens as **secrets** in your CI/CD system's secret manager — never commit them to a repository, even a private one.


API/Web Client
TLS Termination - valid cert, strong ciphers
Rate Limiting / fail2ban
CSRF Token Validation
Authorization Check - RBAC/ACL
Audit Log Entry Written
Action Executed

7. Securing the Web UI & API Endpoints

- Restrict web UI/API access to a **management network or VPN** where possible — don't expose port 8006 directly to the internet
- Enable **fail2ban** (or an equivalent) against repeated failed login attempts
- Review the **audit log** (`/var/log/pve/` , `pvesh get /cluster/log`) regularly, not just after an incident

```
pvesh set /cluster/firewall/options --enable 1 --policy_in DROP
pvesh create /cluster/firewall/rules --action ACCEPT --type in \
  --dport 8006 --proto tcp --source 10.10.0.0/16
```


Self-signed cert - default install
Replace with internal CA or public CA cert
Or automate via built-in ACME client - Let's Encrypt
Automatic renewal before expiry
Monitor expiry + audit log for cert changes
Forward audit logs to central SIEM

8. TLS Certificates & Certificate Management

```
# Use the built-in ACME client for automated cert issuance/renewal
pvenode acme account register default admin@example.com
pvenode acme cert order
pvenode config set --acme domains=pve1.example.com
```

Never leave the **default self-signed certificate** in production — beyond the "insecure site" warning, it trains users to click through TLS warnings, undermining the value of TLS entirely.

9. Auditing User Activity & Permissions

```
# Cluster-wide activity log
pvesh get /cluster/log --limit 200

# Review effective permissions for a specific user
pveum user permissions jsmith@ad-corp

# Periodically review ALL ACL entries for scope creep
pveum acl list
```

Schedule a **quarterly access review**: confirm every user/token still needs the access it has, and remove anything left over from a completed project or departed employee.

10. Lab: Hardening a Proxmox Node

In this lab you will:

1. Harden SSH (key-only auth) and lock down the firewall to a management subnet
2. Enable 2FA for an administrative user
3. Create a custom least-privilege role and apply it via ACL
4. Create a privilege-separated API token scoped to a single VM
5. Replace the self-signed certificate with an ACME-issued one
6. Review the cluster audit log for your own actions

Day 1 — Key Takeaways

- Security is layered — network, host, hypervisor, guest, and data each need dedicated controls
- **2FA + realm-based auth** should protect every administrative account, not just `root@pam`
- RBAC works best with **narrow custom roles**, not broad built-in ones
- API tokens should always use **privilege separation** and their own scoped ACLs
- TLS and audit logging are baseline requirements, not optional extras